REPORT

Zajęcia: Analog and digital electronic circuits
Teacher: prof. dr hab. Vasyl Martsenyuk

Lab 3

Date 03.03.26

Topic: "Digital Filter Design and Analysis: Implementing FIR and IIR filters in Python."
"Adaptive Filtering: Applying adaptive filtering algorithms to noise reduction. "

Variant 13

Kostiantyn Hrekov
Informatyka II stopień,
stacjonarne,
1 semestr,
Gr 1.a

1. Problem statement:

Variant 13: - Design an FIR filter with the following coefficients and implement it in Python to reduce noise in a noisy sinusoidal signal.

$$\text{FIR Filter Coefficients: } b = \{1, 0.5, 0.5\}$$

- Design an IIR filter with the following coefficients and implement it in Python to reduce noise in the same noisy sinusoidal signal.

$$\text{IIR Filter Coefficients: } b = \{1, -0.5, 0.2\},\ a = \{1, -0.3\}$$

- Implement an adaptive LMS filter in Python with a step size $\mu = 0.1$ and filter length $M = 4$ to reduce noise in the same noisy sinusoidal signal.

2. Input data:

b = [1, 0.5, 0.5]
a = [1, -0.3]

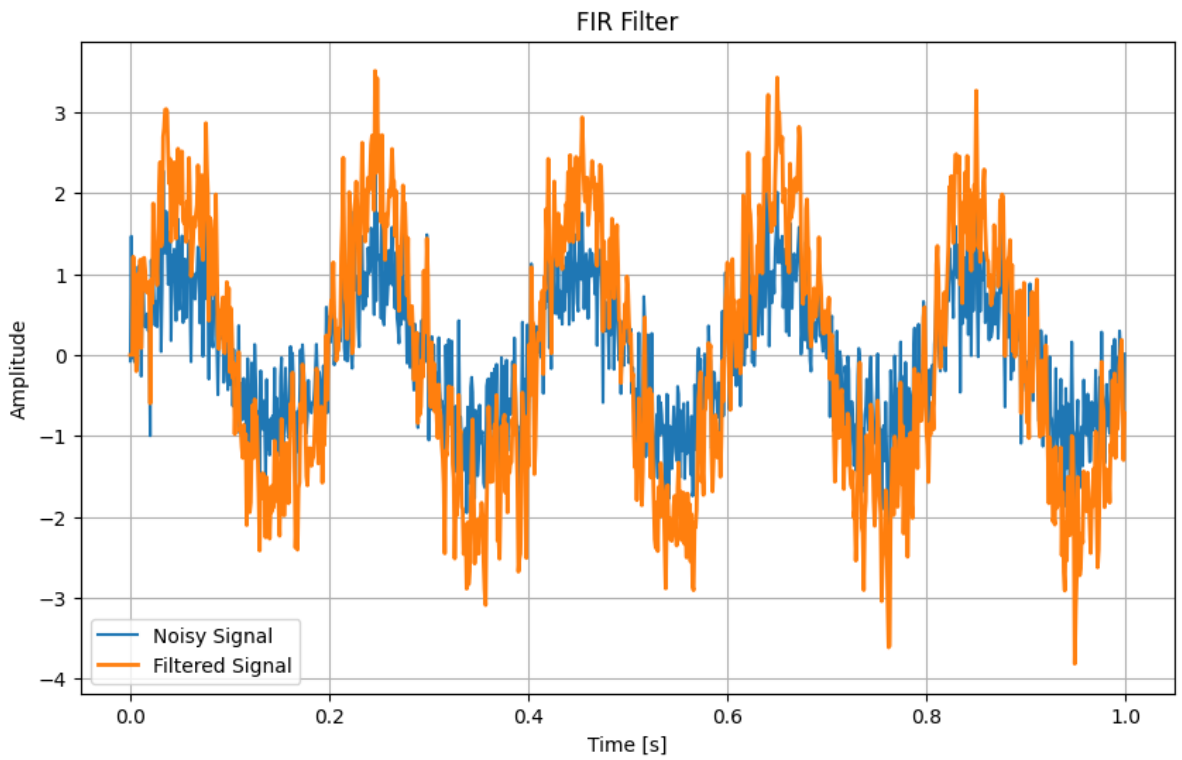
3. Commands used (or GUI):

FIR implementation:

```
import numpy as np
import matplotlib.pyplot as plt


def fir_filter(x, b):
    """
    FIR filter implementation.

    Parameters:
    x : ndarray
        Input signal.
    b : ndarray
        Filter coefficients.

    Returns:
    y : ndarray
        Filtered output signal.
    """
    M = len(b)
    y = np.zeros(len(x))
    for n in range(M, len(x)):
        y[n] = np.dot(b, x[n - M + 1:n + 1][::-1])
    return y


# Example usage and plotting
fs = 1000  # Sampling frequency

t = np.arange(0, 1, 1 / 1000)
clean = np.sin(2 * np.pi * 5 * t)
noise = 0.5 * np.random.randn(len(t))
x = clean + noise
b = [1, 0.5, 0.5]

y = fir_filter(x, b)

plt.figure(figsize=(10, 6))
plt.plot(t, x, label="Noisy Signal")
```

```
plt.plot(t, y, label="Filtered Signal", linewidth=2)
plt.legend()
plt.title("FIR Filter")
plt.xlabel("Time [s]")
plt.ylabel("Amplitude")
plt.grid()
plt.show()
```

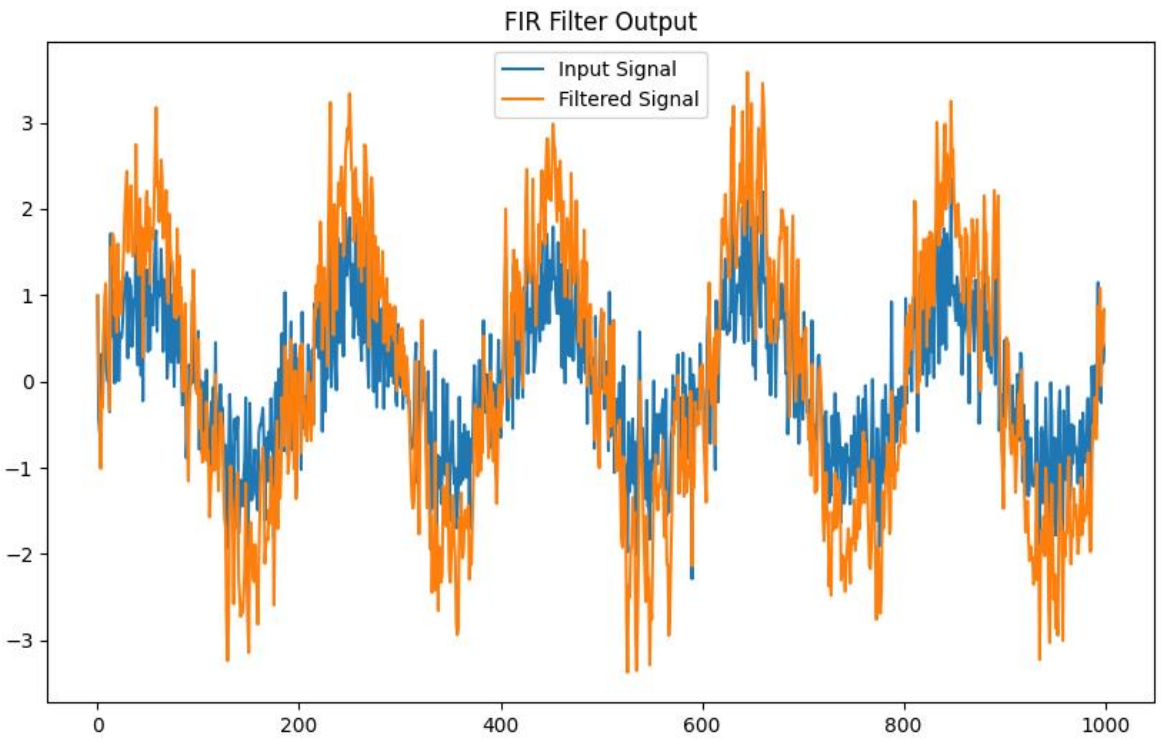
Output:



FIR using scipy.signal library:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import freqz

# Define FIR Filter
def fir_filter(b, x):
    M = len(b)  # Number of coefficients
    y = np.convolve(x, b, mode='full')[:len(x)]  # Apply filter
    return y

# Example FIR Coefficients and Input Signal
b = [1, 0.5, 0.5] # FIR coefficients
clean = np.sin(2 * np.pi * 5 * t)
noise = 0.5 * np.random.randn(len(t))
x = clean + noise

# Filter the Signal
y = fir_filter(b, x)

# Plot
plt.figure(figsize=(10, 6))
plt.plot(x, label="Input Signal")
plt.plot(y, label="Filtered Signal")
plt.legend()
plt.title("FIR Filter Output")
plt.show()
```

Output:

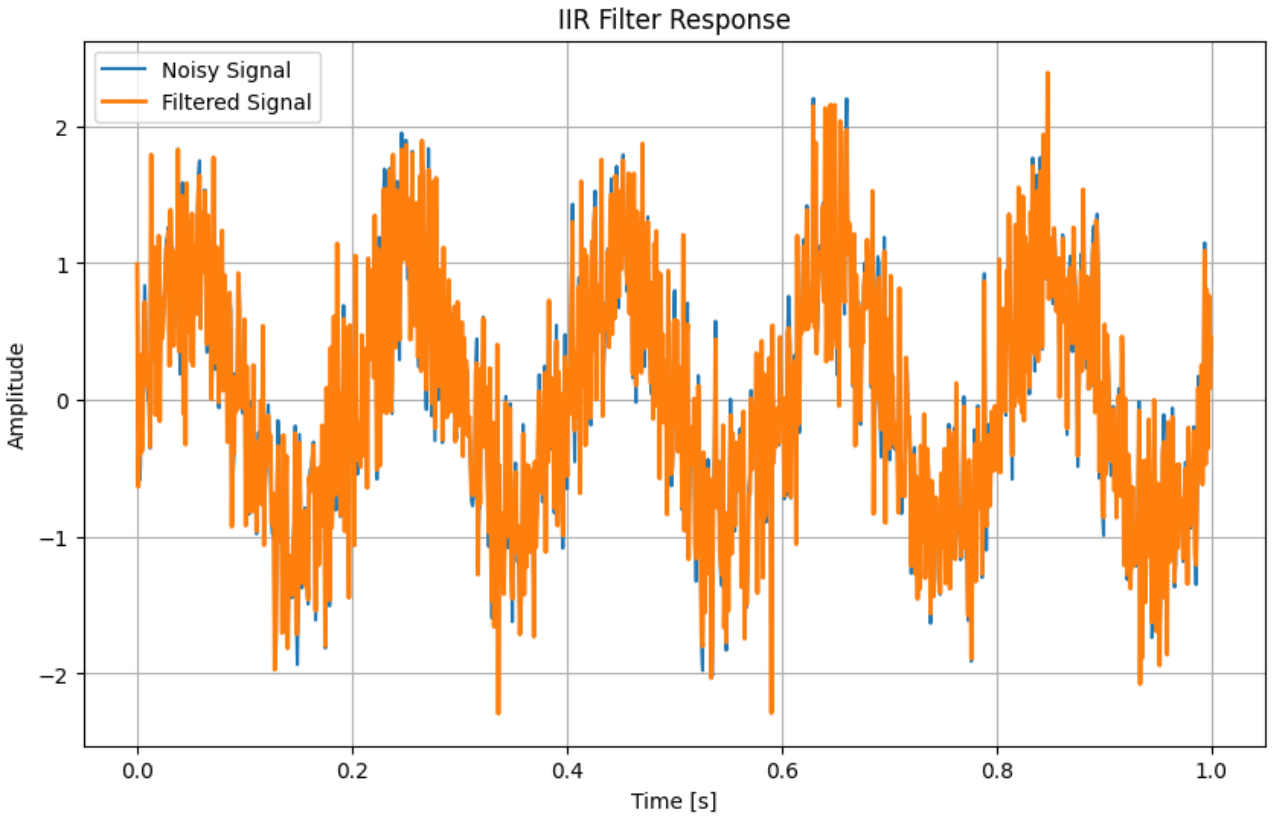


IIR Filter implementation:

```
import numpy as np
import matplotlib.pyplot as plt

def iir_filter(x, b, a):
    """
    IIR filter implementation.

    Parameters:
    x : ndarray
        Input signal.
    b : ndarray
        Numerator coefficients.
    a : ndarray
        Denominator coefficients.

    Returns:
    y : ndarray
        Filtered output signal.
    """
    M = len(b)  # Length of numerator coefficients (b)
    N = len(a)  # Length of denominator coefficients (a)
    y = np.zeros(len(x))  # Initialize output signal array

    # Apply filter to each sample in the input signal
    for n in range(len(x)):
        # Numerator part (feedforward)
        # Ensure we use the correct slice length for the convolution
        x_slice = x[max(0, n-M+1):n+1]  # Input signal slice
        y[n] = np.dot(b[:len(x_slice)], x_slice[::-1])  # Apply reverse
convolution for numerator

        # Denominator part (feedback), skip the first sample
        if n >= 1:
            # Ensure we use the correct slice length for the feedback part
            y_slice = y[max(0, n-N+1):n]  # Output signal slice
```

```
            y[n] -= np.dot(a[1:min(N, len(y_slice)+1)], y_slice[::-1])  #
Apply reverse convolution for feedback

    return y

# Example usage and plotting
# Create a noisy signal (for example, a sine wave with noise)
fs = 1000  # Sampling frequency
t = np.linspace(0, 1, fs)  # Time vector
x = clean + noise  # Noisy signal

# IIR filter coefficients
a = [1, -0.3]  # Denominator coefficients (a_0 = 1 by convention)
b = [1, -0.5, 0.2]  # Numerator coefficients

# Apply the filter to the noisy signal
y = iir_filter(x, b, a)

# Plot the results
plt.figure(figsize=(10, 6))
plt.plot(t, x, label="Noisy Signal")
plt.plot(t, y, label="Filtered Signal", linewidth=2)
plt.legend()
plt.title("IIR Filter Response")
plt.xlabel("Time [s]")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()
```

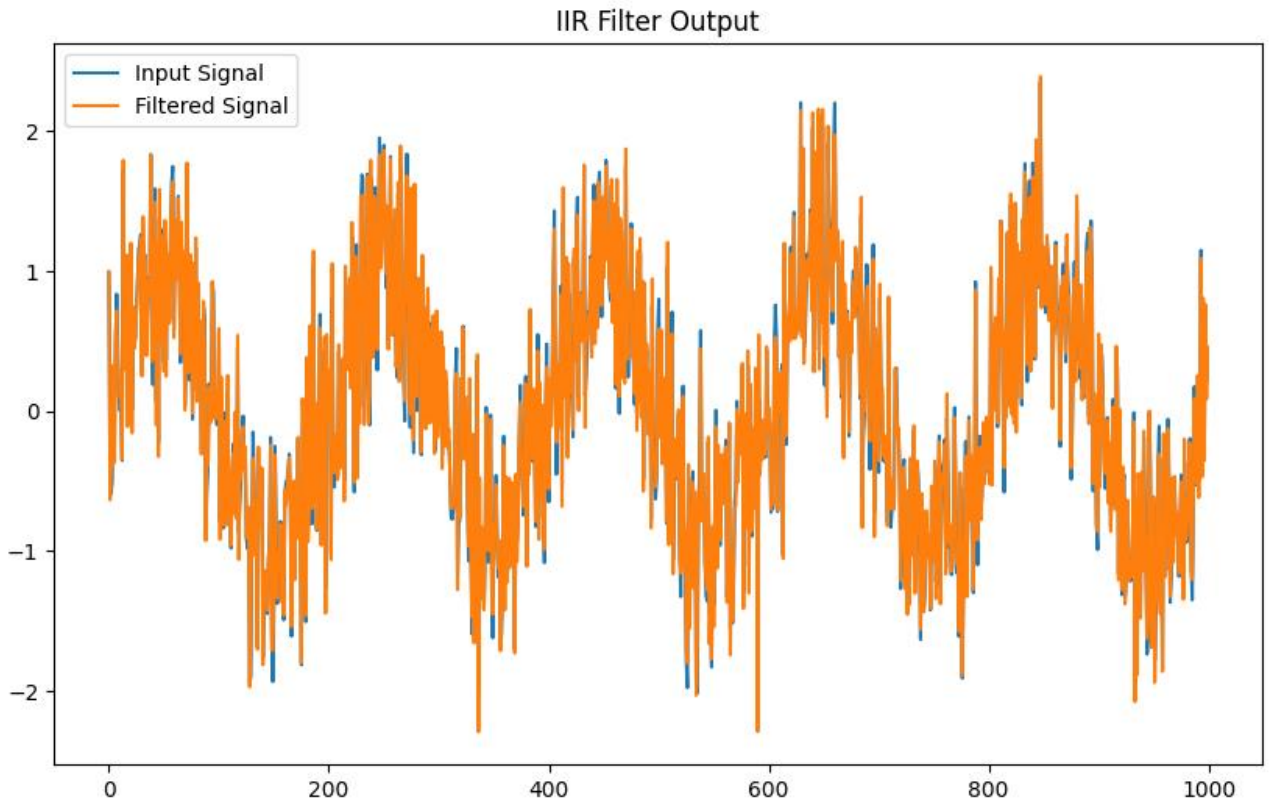
Output:



IIR implementation using scipy.signal:

```
from scipy.signal import lfilter, freqz

# Define IIR Filter Coefficients

b = [1, -0.5, 0.2] # Numerator
a = [1, -0.3] # Denominator


# Example Input Signal
x = clean + noise

# Filter the Signal
y = lfilter(b, a, x)

# Plot the Output
plt.figure(figsize=(10, 6))
plt.plot(x, label="Input Signal")
plt.plot(y, label="Filtered Signal")
plt.legend()
plt.title("IIR Filter Output")
plt.show()
```

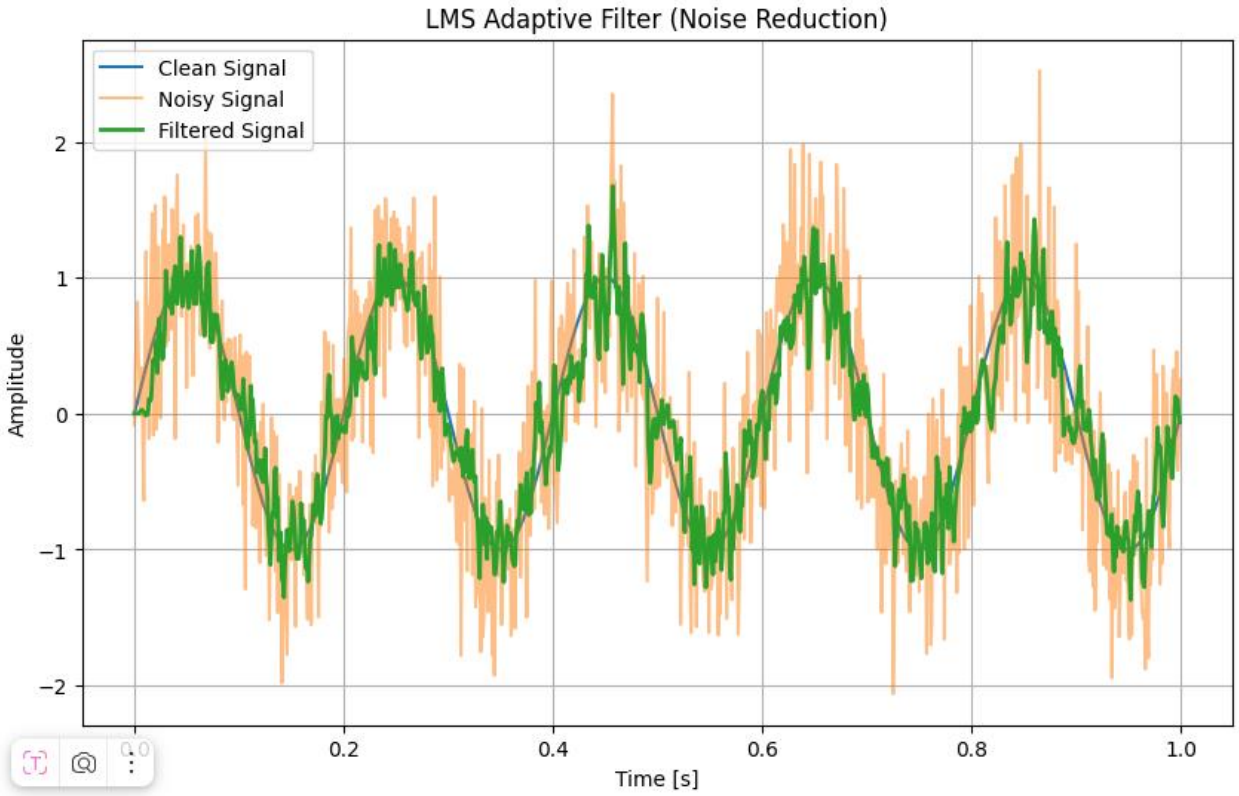
Output:



Adaptive LMS Filter Implementation:

```
import numpy as np
import matplotlib.pyplot as plt


# === LMS FILTER ===
def lms_filter(x, d, mu, num_taps):
    n = len(x)
    w = np.zeros(num_taps)
    y = np.zeros(n)
    e = np.zeros(n)

    for i in range(num_taps, n):
        x_segment = x[i - num_taps:i][::-1]
        y[i] = np.dot(w, x_segment)
        e[i] = d[i] - y[i]
        w += mu * e[i] * x_segment

    return y, e, w


t = np.arange(0, 1, 1 / 1000)
clean = np.sin(2 * np.pi * 5 * t)
noise = 0.5 * np.random.randn(len(t))
x = clean + noise  # noisy input
d = clean  # desired (czysty sygnał)

mu = 0.1
num_taps = 4

y, e, w = lms_filter(x, d, mu, num_taps)

plt.figure(figsize=(10, 6))
plt.plot(t, clean, label="Clean Signal")
plt.plot(t, x, label="Noisy Signal", alpha=0.5)
plt.plot(t, y, label="Filtered Signal", linewidth=2)
plt.legend()
plt.title("LMS Adaptive Filter (Noise Reduction)")
plt.xlabel("Time [s]")
plt.ylabel("Amplitude")
plt.grid()
plt.show()
```

Output:



Conclusion:

Due to this lab, we learned how different digital filters - FIR, IIR, and adaptive LMS - affect noise reduction in a sinusoidal signal. We observed that FIR filters provide stable and straightforward smoothing based on fixed coefficients, while IIR filters achieve stronger filtering effects with fewer coefficients but introduce feedback. We also saw that the LMS adaptive filter adjusts its weights dynamically based on the error signal, allowing it to effectively learn and reduce noise over time. This highlights the trade-off between simplicity, efficiency, and adaptability in practical signal processing applications.

Link to remote repozytorium (e.g. GitHub):

https://github.com/KostiaGrekov/DSPja/tree/main/DSPja%202